

MEDIASTRAY - Eloy Rico Payá

MEDIASTRAY

Technical documentation - Eloy Rico Payá 2ºDAW

Profesor: Juan Carlos Gomez Vicente

Última fecha de modificación: 01/06/2026



Summary

Mediastray es una plataforma online de videojuegos en la que los usuarios pueden registrarse para publicar sus propios videojuegos y jugar a los de otros usuarios. Se pueden descargar los juegos tanto para ejecutarlos localmente como ejecutarlos directamente en la página usando la tecnología WebGL. Además, también incluye características de red social como poder seguir juegos y usuarios, sistema de comentarios y reportes, personalización avanzada, etc. La plataforma consta tanto de backend como de frontend siendo ambos desplegables con Docker teniendo, además, cuatro bases de datos para funcionalidades distintas

Mediastray is an online gaming platform where the users can register to publish their own games and play the other user's ones. The games can be downloaded for executing them locally as well as playing them in the very website using the WebGL technology. Also the platform includes social network features such as being able to follow games and users, a comments and reports system, advanced customization, etc. The platform consists in a Backend and a Frontend, being both of them deployable with Docker having also four databases for different use cases.

Index

Index

1 Introduction.....	4
1.1 Context and presentation.....	4
1.2 Project's objectives.....	4
1.3 Project management.....	5
1.4 Access to the repository.....	5
1.5 Access to the deployed app.....	5
2 Tecnologías y arquitecturas empleadas.....	6
2.1 Guía de estilo.....	6
Tecnologías empleadas en el diseño.....	6
Justificación del estilo elegido.....	7
Colores.....	7
Tipografías.....	8
Disposición y espaciado.....	8
Multimedia.....	8
Mapa de navegación.....	9
Diseño responsive.....	10
Logos.....	11
Ejemplos del diseño final.....	12
2.2 Listado y justificación de las herramientas en backend.....	14
Tecnologías en el backend.....	14
Arquitectura empleada.....	14
Documentación de la API.....	15
2.3 Descripción y justificación de la base de datos.....	16
Tecnologías y esquema Polyglot.....	16
Diagrama UML.....	17
Definición de entidades.....	17
2.4 Listado y justificación de las herramientas en frontend.....	23
Tecnologías en el frontend.....	23
Arquitectura empleada.....	23
2.5 Servicios utilizados y proceso de despliegue.....	25
Arquitectura de microservicios.....	25
Funcionalidades CI/CD y automatizaciones.....	25
Tecnologías de despliegue.....	25
Pasos para desplegar.....	26
Despliegue actual.....	27
Monitoreo y testing.....	27
2.6 Herramientas externas para el desarrollo.....	28
3 Guía de la app.....	29
3.1 Validaciones de campos.....	29
3.2 Cuentas Magna.....	29
3.3 Publicación de juegos.....	29
3.4 Moderación y normas.....	29
3.5 Algoritmo de recomendación.....	29

4 Ampliaciones y líneas de mejora.....	30
5 Conclusiones y comentarios.....	31

Content

1 Introduction

This is the documentation of the Mediastray app, also called 'Memoria de TFG', this document encompasses extended contents for its presentation as well as documentation at the user and developer level, covering themes like the API endpoints, how to use it, technologies used, etc.

1.1 Context and presentation

This project idea came from the inspiration of other platforms such as Itchio, Gamejolt, Steam and other online gaming platforms. I wanted a social-media-like app that allowed users to publish their games so they can play them directly on the site, doing strong emphasis on indie games from small studios or even singular people that don't have the resources to hire a publisher nor the knowledge to do it themselves.

The key feature of the app is that if these games are published in the WebGL format they can be played directly on the web. The main reason casual players wouldn't try games of this size (indies or even bellow that) is that they would have to download and install them (which takes space and time), this way the effort for quickly trying a game and some other concerns (such as players not trusting an executable game) get solved.

Before aiming the project to this, I had some other ideas that could fit well but got discarded anyways, such as: a hub where people could post information of their lost pets with map location so other people can help them, a text editor for writers where they could add automatic sounds and soundtrack to specific parts of their books and publish them online, an ultra private messaging app that uses P2P and obfuscation so war activists or citizens of authoritarian regimes can communicate seamlessly.

1.2 Project's objectives

The project had some initial objectives (some of them were added during development) and, even tough some of them couldn't be accomplished, the most crucial ones are fully implemented.

The objectives that got finished were: full secure user system with log in and profiles and user followings, game publishing with game information and files to download or play in the web, a comment system with hierarchy so each comments can have responses, multi language support, moderation system with admin panels and reports, responsive design for mobile phones, several security and optimization features.

However, the objectives that couldn't got finished either for time or money limitations were: actual lasting deployment in a cloud infrastructure, mail system for user verification/password change or notifications, forum system that gets attached to a game and has comments inside, a working app for Linux, Android and Windows, some security and optimization features that couldn't be added, game versioning and Game Jams, a game-attached API for achievements and cloud save, multimedia hosting, more sophisticated monitoring system, custom HTML pages for each game, actual working payment methods.

The features that couldn't be added in the given time span might seem many, but most of them are either secondary or very hard to implement (or cost money)

MEDIASTRAY - Eloy Rico Payá

1.3 Project management

The project repository was (and will be) hosted in GitHub during all its development and there were two Git branches, main for the already working code and dev for a safe cloud save and version management on GitHub. If some CI/CD feature was to be added here, it would be on the main branch. This method was more fit than other ways (such as one branch per person or per task) because of the size of the team (only me)

About the time management, the app first had a design phase where the main systems were pre-documented before implementing, and then there were cycles of development for each feature. For example instead of doing the full backend first and the full frontend then, a feature (lets say users) was first designed, then implemented into the backend, tested, implemented into the frontend and tested again, before continuing to the next feature (games in this case) And after completing the designing and development phases then came the testing & bug fixing one, and finally deployment and documentation.

1.4 Access to the repository

Here is the link to the source code of the GitHub repository of Mediastray, however only the main branch is public in order to avoid private data leaks

<ENLACE>

1.5 Access to the deployed app

Here is the current working link to access the deployed app, there will be an specific section about deployment in this document. The app will stop working around 01/07/2026 because it was a test deployment. If this is getting read during the deployment life span, feel free to make an account and use the app.

<ENLACE>

<CREDENCIALES ADMIN>

2 Tecnologías y arquitecturas empleadas

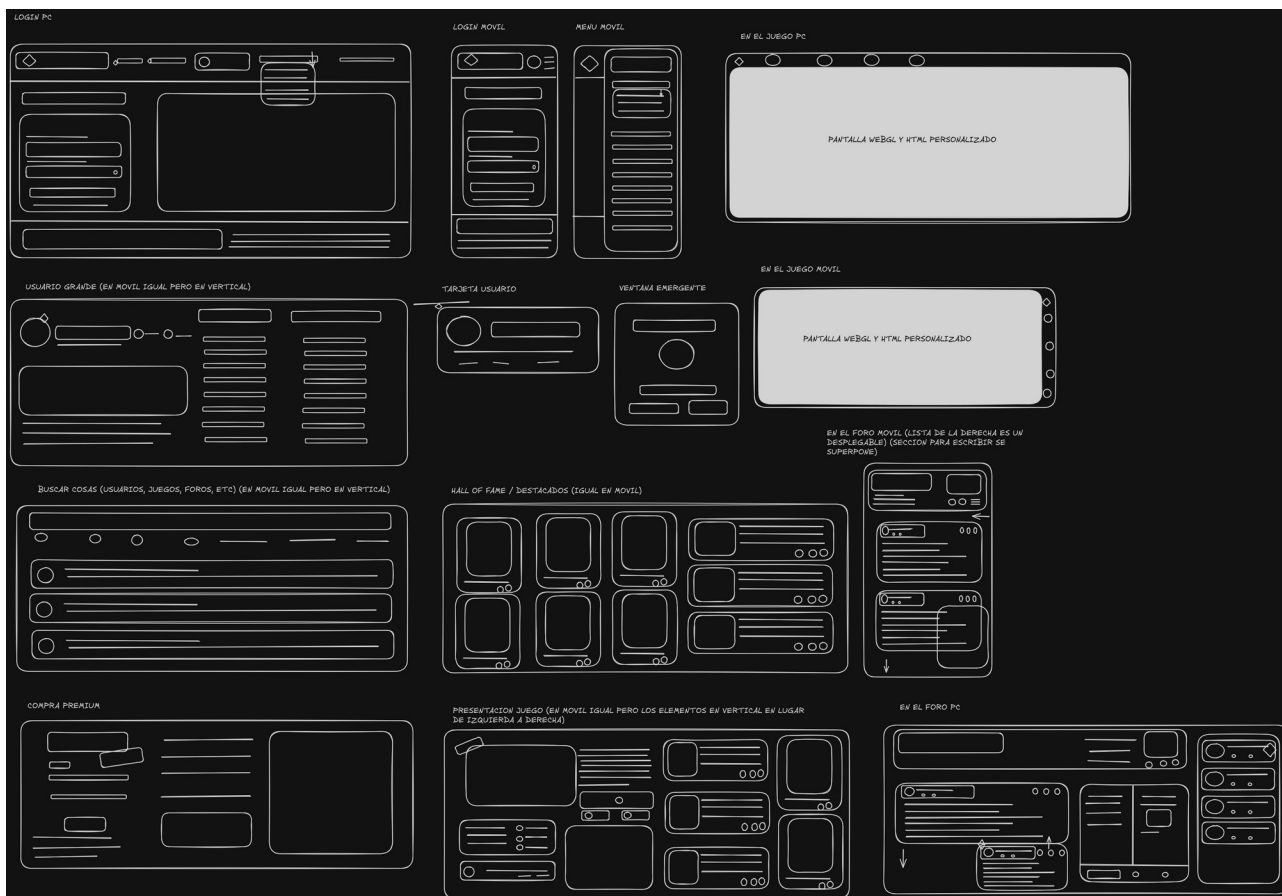
Como el objetivo principal del proyecto era el autoaprendizaje, se intentó usar la mayor cantidad de tecnologías y librerías posibles dentro de lo aceptable, quizás en un entorno real se hubiesen usado otras tecnologías, en menor cantidad o más privadas. De todas formas no se usó un stack ya existente (como MERN o LAMP) sino que se eligieron las herramientas, librerías y tecnologías manualmente en base a los requisitos (y optando siempre que se pueda por opciones open-source).

2.1 Guía de estilo

Tecnologías empleadas en el diseño

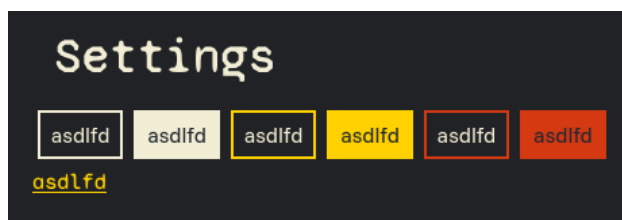
Como tecnologías para estilizar el frontend se usaron CSS raso y Tailwind (ya que encaja bastante con el modelo de componentes de React). También se planteó usar otras tecnologías como SASS o Bootstrap.

En cuanto a los mockups, se hicieron principalmente con Excalidraw y Penpot, aquí hay se encuentran los prototipos de bajo nivel (se hicieron antes del desarrollo de la app así que el resultado final no es exactamente igual)



Prototipos de bajo nivel de la interfaz

Apenas se usaron prototipos de alto nivel, en su lugar se fue iterando haciendo pruebas con distintos estilos en Tailwind, aquí está el resultado de una de estas pruebas



Prototipo de alto nivel básico

Justificación del estilo elegido

Se optó por un diseño vibrante, llamativo y "juguetón" usando colores muy vivos con un alto nivel de resalte. Se puede observar que el tono principal de la aplicación es el amarillo, que simboliza juventud y diversión, muy apropiado para una página de videojuegos.

También se optó por un diseño muy poco redondeado, con esquinas totalmente cuadradas y símbolos en estilo pixelart, se tomó esta decisión ya que la página está enfocada en juegos indie los cuales suelen tener estética pixelada, además que por lo general un estilo muy redondeado si bien da la sensación de confianza puede parecer que uno no se encuentra en un entorno muy personalizable, sin embargo las páginas con estilos cuadriculados suelen ser las que en términos de diseño dan la sensación al usuario de que se encuentra en un entorno con muchas libertades de personalización y configuración, lo cual podría resultar atractivo a simple vista para los desarrolladores.

Colores

Se partió de una paleta de colores ya existente y se modificó un poco, también para añadir más variedad. Los colores en uso actualmente son:

- Color de fondo **#202225**
- Color de fondo secundario: **#333435**
- Color de fondo terciario: **#222831**
- Color de fondo auxiliar: **#848383**
- Color principal (usado en la tipografía): **#f2edd5**
- Color de resaltado (de enlaces y botones) (sería también el color principal de la app): **#ffd100**
- Color de error: **#d53912**
- Color auxiliar (usado para secciones relacionadas con Magna): **#7b2bcf**
- Color auxiliar secundario (usado en algunos mensajes de éxito): **#7de62b**

A continuación se muestra un ejemplo de como se verían todos estos colores juntos (esta imagen es uno de los prototipos de bajo nivel), esta imagen también representa el diseño sin redondeado antes justificado.



Ejemplo de composición de colores

Hay que tener en cuenta que esta imagen se hizo antes de desarrollar el diseño completo y solo representa los colores y el "filo en las esquinas"

MEDIASTRAY - Eloy Rico Payá

Tipografías

Se usaron exactamente 5 fuentes todas ellas provenientes de Google Fonts y usadas para propósitos distintos

Funnel Sans: fuente principal de la aplicación, usada en los textos

Lexend Deca: usada también de manera frecuente, sobretodo en elementos interactivables

Syne Mono: usada en los títulos y en los elementos de la cabecera

Chelsea Market: usada principalmente de manera auxiliar

Datatype: usada principalmente en secciones más técnicas ya que es monoespaciada

Alternativamente se pensó en usar un tipo de fuente adicional estilo pixelart, pero daba problemas para el diseño responsive así que ~~SE DESARROLLÓ UNA FUENTE PROPIA CON CALLIGRAPHY QUE SERÍA RACKWRITE~~, pero terminó quedando descartada por su disonancia con el resto del diseño.

En cuanto a los tamaños, se usó la **negrita** para partes importantes, la *cursiva* para literales o nombres y el subrayado para enlaces.

Y en cuanto a los tamaños, el estándar para los textos y elementos es de 13pt, mientras que los títulos

usan los tamaños se usa **27pt** para los títulos principales h2, **22pt** para los secundarios h3 y **18pt** para los terciarios h4. Los títulos se usan de manera jerárquica, teniendo h2 como principal.

Disposición y espaciado

En cuanto a parámetros como espaciado entre letras, palabras, líneas o párrafos se usaron los que vienen por defecto al usar Tailwind que, en teoría, sería un espaciado de líneas de 21pt, un margen de 15pt para los párrafos, y los espaciados de palabra y letra establecidos en "normal".

En cuanto a los márgenes y paddings de otros elementos (como secciones, botones o inputs) estos pueden variar dependiendo del contexto (por ejemplo botones similares aparecerán más juntos), pero por estándar se usa un margen de 5px y un padding de 3px (aunque los inputs suelen tener un padding de 6px para que no queden muy compactos los formularios)

Se recomienda usar los elementos de React ya existentes (como íconos, botones o enlaces) para extender la interfaz si fuese necesario

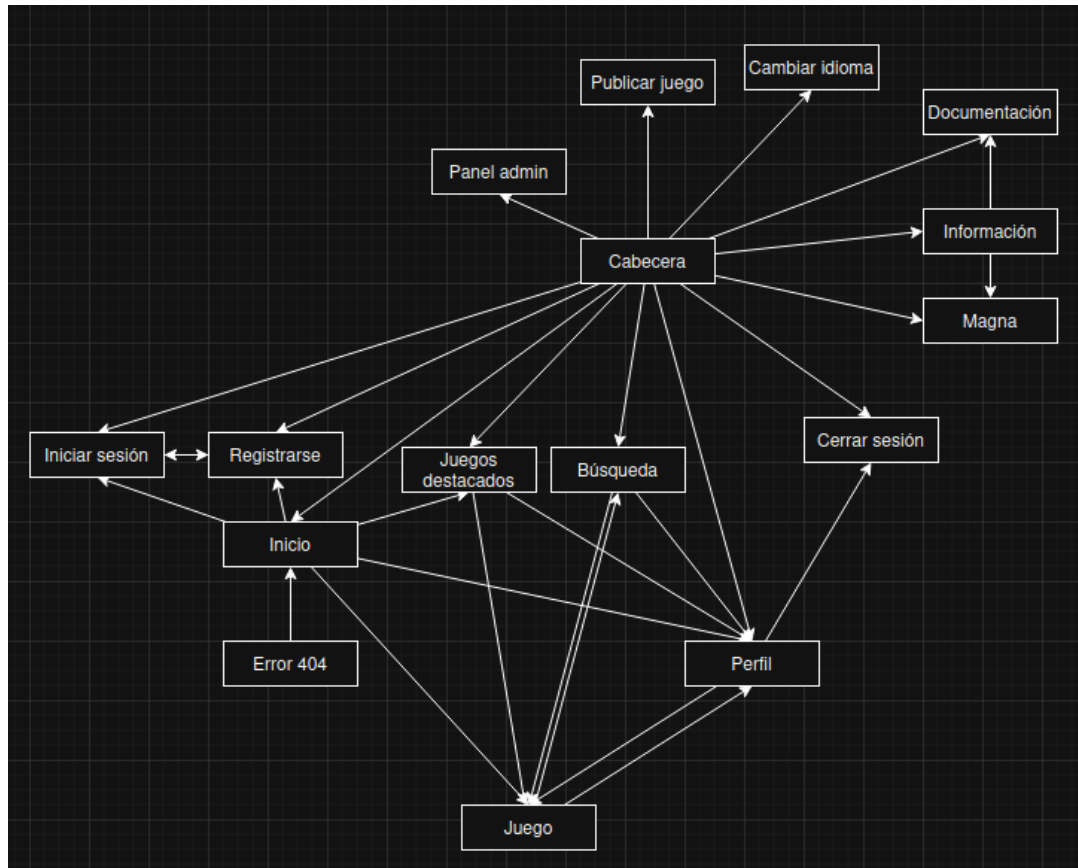
Multimedia

En algunas partes de la app podemos encontrar multimedia incrustada como imágenes o videos, en cuanto a las imágenes las hay de dos tipos, las que sirven como decoración para la interfaz (como los íconos o los logos) o las que vienen por parte del usuario (como un carrusel de imágenes en un juego), en el segundo caso, se establece un tamaño máximo concreto (dependiendo del contexto en el que se utilice) y que cuando se clique que lleve al origen de la imagen.

En cuanto a los videos (por ejemplo los trailers de los juegos insertados mediante URL o Youtube) pasa algo muy similar, y se permite que aparezcan los controles de video.

Mapa de navegación

A continuación se muestra el mapa de navegación de la web, se intenta que sean necesarios el mínimo de clics para llegar a cualquier parte (hay que tener en cuenta que la cabecera está siempre presente)



Mapa de navegación de la app

MEDIASTRAY - Eloy Rico Payá

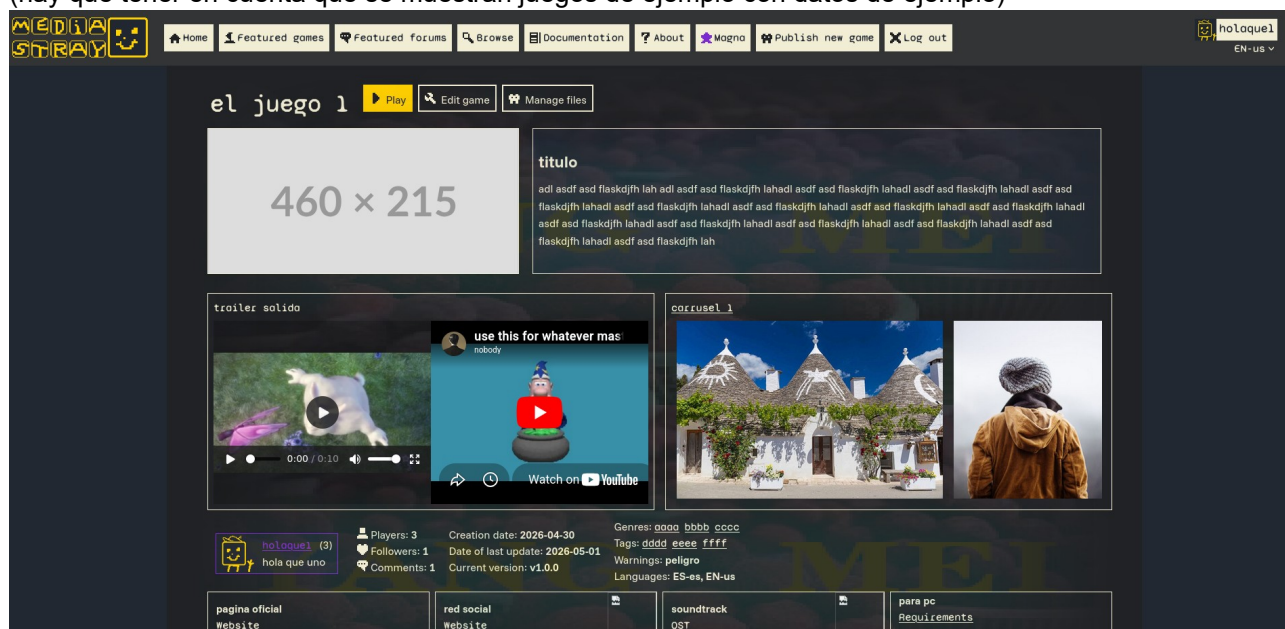
Diseño responsive

Si bien la página está enfocada a clientes con una pantalla más ancha (como pc o tablet), el diseño también admite pantallas verticales, así que los navegadores móviles también pueden navegar por la app e incluso jugar a los juegos (si estos admiten también controles táctiles)

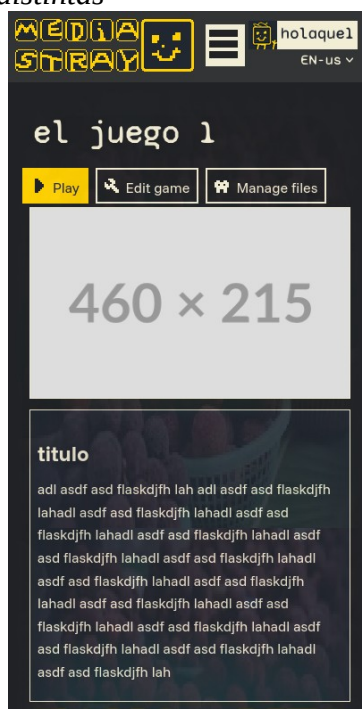
Para el diseño responsive se usa una técnica muy sencilla, la mayoría de partes de la aplicación constan de varias columnas (típicamente 2) y estas tienen unos márgenes para alejarse de los bordes, cuando se visualiza la página en una pantalla vertical estas columnas pasan a renderizarse una debajo de otra y reducen sus márgenes exteriores (a 3px normalmente), permitiendo una adaptabilidad sencilla. En el caso de la cabecera esta muestra solo los elementos indispensables y muestra el resto en un menú hamburguesa.

En el sentido contrario (resoluciones más grandes como 4K) se aprovecha que el tamaño del texto se mide en unidades relativas para que las partes de la aplicación no se vean demasiado pequeñas, además se usa un diseño de columna central que solo es visible a partir de resoluciones más grandes.

A continuación se muestra un ejemplo de la misma página siendo renderizada en resoluciones diferentes (hay que tener en cuenta que se muestran juegos de ejemplo con datos de ejemplo)



Vista de la app en dos resoluciones distintas



Logos

A continuación se muestra el logo de la aplicación en distintos formatos, como se observa sigue siendo pixelart y usa los colores de la paleta antes mencionada



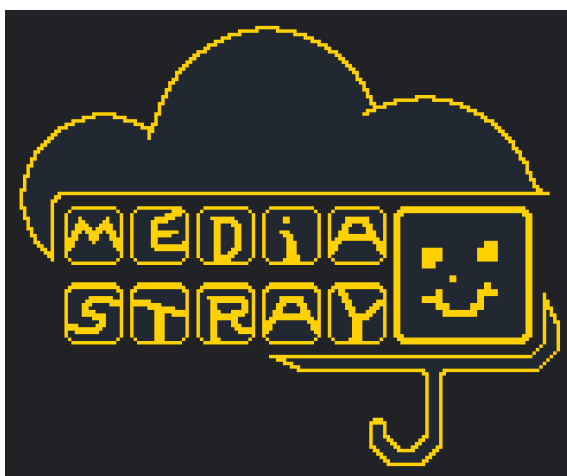
Isotipo



Imagotipo 1



Imagotipo 2



Isologo 1

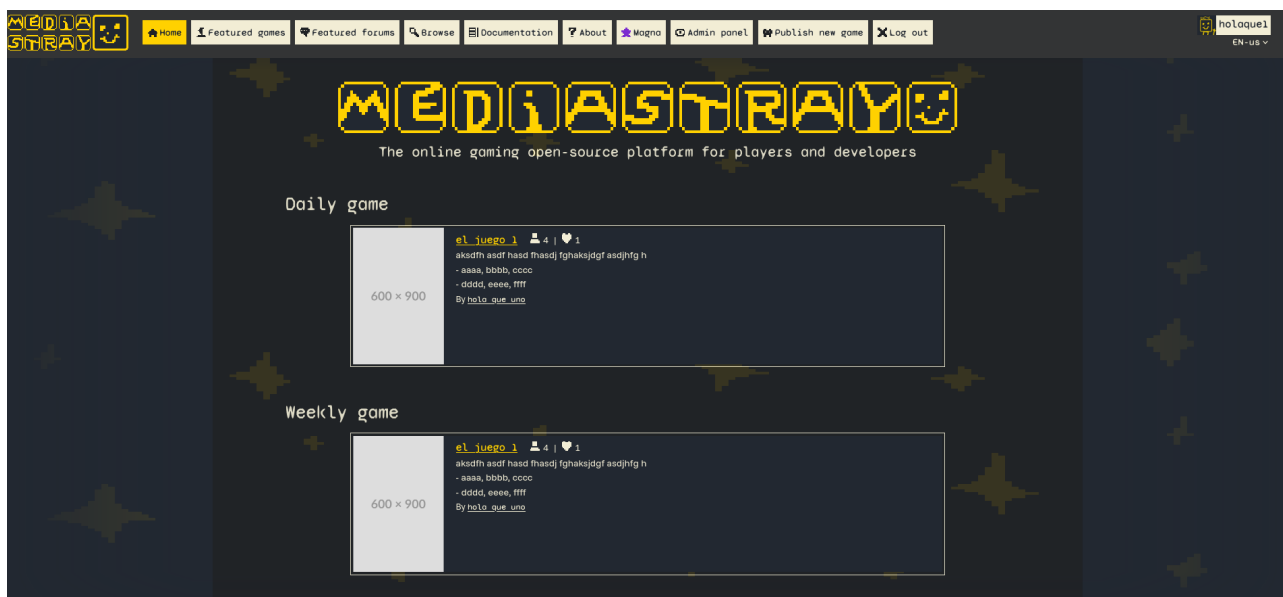


Isologo 2

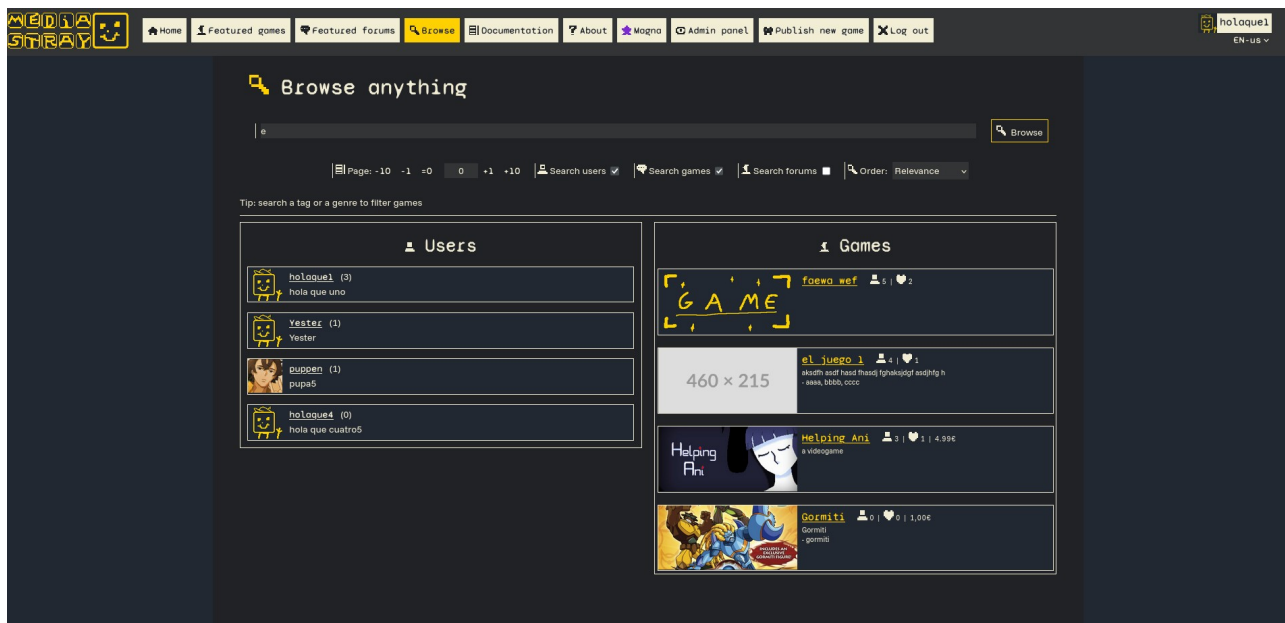
MEDIASTRAY - Eloy Rico Payá

Ejemplos del diseño final

A continuación se muestran ejemplos del diseño final de la app



Página de inicio



Página de búsqueda



Página del perfil

MEDIASTRAY - Eloy Rico Payá

MEDIASTRAY

holaque1
EN-us

Create game

Fields marked with (*) are required. Have also into account that the game will be uploaded as private, you will be able to publish it once it's created, as well as adding game files and some extra info.

(*) Title

Mi juego

Game cover URL 1 (460x215px)

<https://coversforgames.com/images/1/460x215>

Game cover URL 2 (600x900px)

<https://coversforgames.com/images/1/600x900>

Game cover URL 3 (1920x1080px)

<https://coversforgames.com/images/1/1920x1080>

Current version:

The version's tag must have less than 16 characters and be formed by digits and letters

Short description

Description (Markdown format will be rendered to the public only if user has Magna account):

hola

- uno

hola

- uno
- dos

Minimum age (0 for unrestricted)

Página de crear juego

MEDIASTRAY

holaque1
EN-us

Home

Featured games

Featured forums

Browse

Documentation

About

Magna

Admin panel

Publish new game

Log out

Featured games

Here are shown the most popular games in the platform based on players, followers, comments and other features.

Page: -10 -1 =0 0 +1 +10

GAME

faewo wef

5 | 2

By [hola_que_uno](#)

600 x 900

el juego 1

4 | 1

aksdhf asdf hasd fhasdj fghaksjdgf asdjhtg h

- aaaa, bbbb, cccc

- dddd, eeee, ffff

By [hola_que_uno](#)

GAME

Helping Ani

3 | 1 | 4.99€

a videogame

- indie, terror, horror

By [lnlnta](#)

Página de juegos destacados

2.2 Listado y justificación de las herramientas en backend

Tecnologías en el backend

Para el backend se utilizaron múltiples tecnologías, como runtime principal se usó NodeJS usando Typescript, estas son las librerías empleadas y por qué

- **Bcrypt**: Usado para el almacenamiento de hash de contraseña
- **Cors**: Usado para tener opciones extendidas de políticas de CORS en el servidor
- **Dotenv**: Usado para recopilar las variables del archivo .env
- **DrizzleORM**: Usado para conectar con la base de datos relacional en lugar de escribir las consultas SQL
- **Express**: Usado para tener facilidades al desplegar un servidor HTTP
- **FormData**: Usado para recibir datos en el formato FormData que admite archivos en red
- **JSONWebToken**: Usado para generar tokens con el algoritmo JWT usados para las sesiones
- **Mongoose**: Usado para conectar con la base de datos MongoDB, que además actuaría como ORM
- **Multer**: Usado para operaciones más avanzadas de archivos
- **PG**: Usado para conectar con la base de datos de PostgreSQL
- **Postgres**: También usado para conectar con la base de datos de PostgreSQL
- **PromClient**: Usado para mostrar métricas al servicio de Prometheus+Grafana
- **Redis**: Usado para conectar con la base de datos de Redis
- **Stream**: Usado para ciertas operaciones de archivos
- **Unzipper**: Usado para descomprimir archivos .zip
- **UUID**: Usado para generar UUIDs
- **Zod**: Usado para tener funciones de validación

Como suele ser común, se usó el gestor de paquetes NPM y package.json (en el cual también hay algunos comandos preestablecidos para automatizaciones). A continuación se muestran más librerías que solo se emplean durante el desarrollo

- **@types**: No es una librería sino un grupo de librerías, agrega compatibilidad con los tipos de TypeScript
- **Concurrently**: Usado para ciertas operaciones de automatizado y CI/CD en el propio package.json
- **Nodemon**: Usado para que la app se reinicie cuando cambie el código en el entorno de desarrollo
- **TSNode, TSX, TypeScript, Esbuild**: Dependencias de TypeScript

Arquitectura empleada

Se ha empleado una arquitectura de capas (muy similar a MVC). En la capa más externa se abren los endpoints de la API, los cuales reciben y parsean la información de la petición para mandarla a la siguiente capa, que sería el controlador.

En la capa del controlador estarían todas las funciones relacionadas con las entidades de la aplicación, recibiendo los datos necesarios por parámetros y en ocasiones el ID del usuario que hizo dicha petición, estas funciones devolverían el resultado de dicha operación de nuevo al endpoint para devolver la respuesta.

MEDIASTRAY - Eloy Rico Payá

La capa más al fondo sería la del modelo, objetos con estándares para cada entidad con los cuales se realizan las operaciones de la capa de controlador.

Además cada ruta hace uso de Middlewares que validan cosas como que la petición tenga el API_TOKEN, que el SESSION_TOKEN pertenezca a un usuario y cumpla los permisos exigidos, etc.

A continuación se muestra un esquema explicando el proceso de como este backend procesaría una petición.

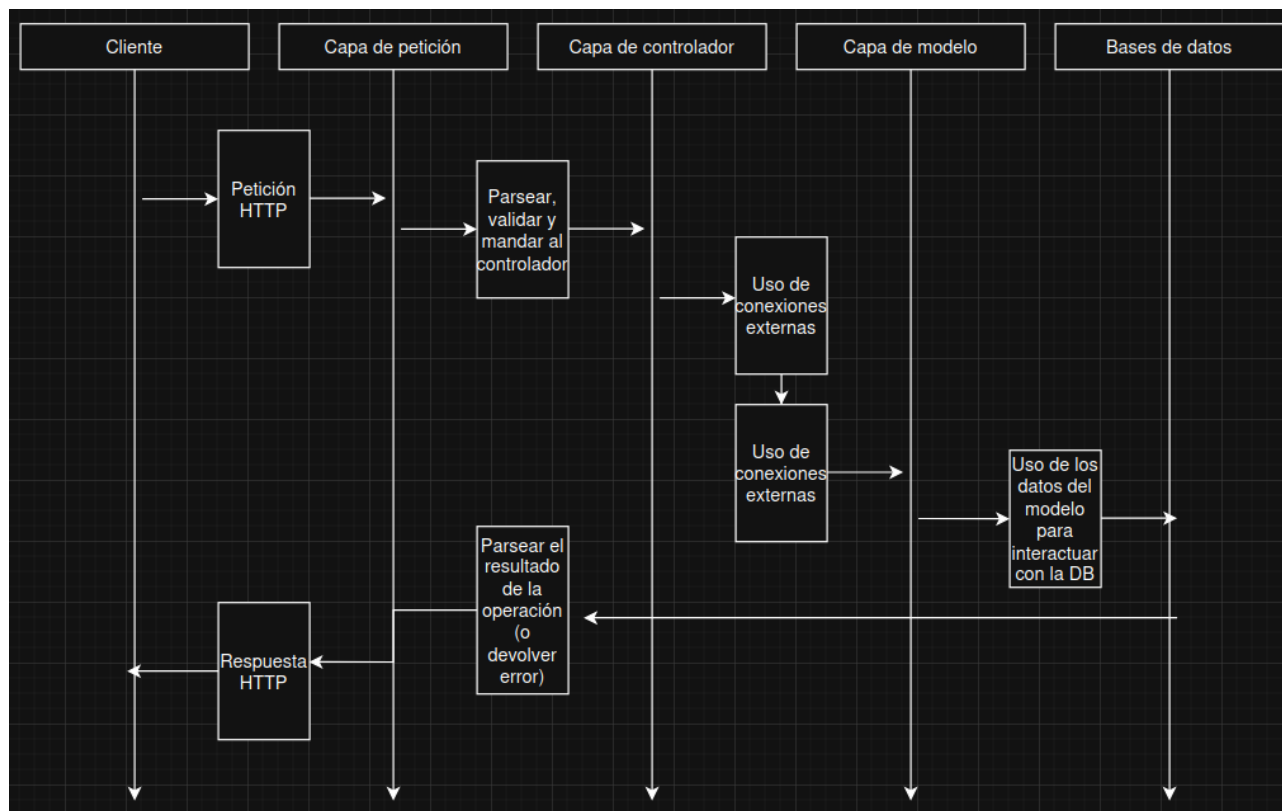


Ilustración de la arquitectura empleada en el backend

El backend también tiene implementado un script de pruebas activable para probar las conexiones (bases de datos, archivos, logs, etc), un sistema de logs propio que almacena información en ciertos archivos y un servidor secundario para dar las métricas a Prometheus y Grafana. Más información relacionada con esto se puede encontrar en el apartado de despliegue.

Documentación de la API

Esta es la documentación de la API (explicación de cada endpoint, que recibe y que devuelve), véase también la definición de cada entidad en el apartado siguiente.

No se incluye en este documento porque se haría demasiado largo, en su lugar se creó la documentación automáticamente con Bruno, se puede acceder desde el repositorio abriendo docs/API.html. Conviene mirar los ejemplos y los comentarios del código para más información.

<HACER>

Por otra parte, las peticiones necesitan la cabecera X-auth-api con la API key para acceder a endpoints ofuscados y la cabecera X-session-token con el token de sesión para identificar al usuario.

2.3 Descripción y justificación de la base de datos

Tecnologías y esquema Polyglot

Como el objetivo del proyecto era el aprendizaje, se usaron 4 bases de datos distintas, aunque en entornos reales con apps grandes esto es más común de lo que parece. Concretamente se usaron estas bases de datos

- **PostgreSQL**: Usado para los usuarios y los juegos (y los foros si se hubiesen empleado) ya que son las entidades principales de la base de datos y requieren una validación estricta y una estructura sólida. A esta base de datos se accede mediante DrizzleORM

- **MongoDB**: Usado para entidades con una estructura más difusa y dinámica, y menos prioritarias, como los comentarios o los objetos intermediarios que indican que un usuario ha seguido a otro, likes, etc.

- **Redis**: Usado como base de datos de caché clave-valor para almacenar datos de las sesiones o información estática accedida frecuentemente, como el juego semanal (que va a ser el mismo en toda la semana y no hace falta recalcularlo cada vez)

- **Archivos**: No se usa un software concreto (mas allá de NodeJS y Multer), pero la app sí necesita almacenar archivos (como los juegos), está diseñado para ser desacoplable y usar otro almacenamiento como el S3 de AWS

De esta manera se aprovechan bases de datos distintas con objetivos distintos para cubrir las distintas necesidades de la aplicación. Lo malo de hacerlo de esta manera son las relaciones entre objetos de Postgres y Mongo, por ejemplo para borrados en cascada, es por eso que la base de datos se diseñó a propósito pensando en este problema para solventarlo.

No se han aplicado las Normas Normales del todo ya que si bien fortalecen la estructura de la base de datos, llega un punto en el que no es rentable en términos de rendimiento, las consultas se harían muy complejas y tardaría más.

Por otra parte los objetos en MongoDB pueden hacer referencia a otros objetos, no es una relación formal como las de SQL pero sí que se almacena su UUID y se valida.

En la base de datos se guardan algunas fechas (cumpleaños del usuario, caducidad del Magna, fecha de creación, etc). Lo que se guarda realmente es el Timestamp en String en formato Unix Epoch (cantidad de tiempo transcurrido desde el 01/01/1970)

Como clave primaria se usan UUIDs en todas las entidades, concretamente su versión 4, así que se almacenan cadenas alfanuméricas de 36 caracteres con guiones.

Los campos en la base de datos requieren validación, pero dicha validación no se hace implícita en la base de datos sino que se evalúa el dato antes de insertar o actualizar.

En general el esquema de base de datos intenta evitar las relaciones N:N o relaciones triples, además de funcionar de una manera jerárquica, en el sentido de que hay una entidad principal que no depende de nada (usuarios) y de esta van dependiendo las otras entidades (juegos, sesiones, comentarios, etc)

MEDIASTRAY - Eloy Rico Payá

Diagrama UML

Aquí se muestra el diagrama UML de las entidades de todas las bases de datos (indicando en qué base de datos se almacenaría) El diagrama se encuentra en el repositorio en formato DrawIO para una mejor vista ya que podría no caber bien dentro de este documento.

FK = ForeignKey, UK = UniqueKey, PK = PrimaryKey, N = Nullable, <pg> = almacenado en Postgres, <mongo> = almacenado en MongoDB, <redis> = almacenado en Redis, <File> = almacenado como archivos. -s = string, -n = número, -b = booleano, -o = objeto, C = borrado en cascada en la relación.

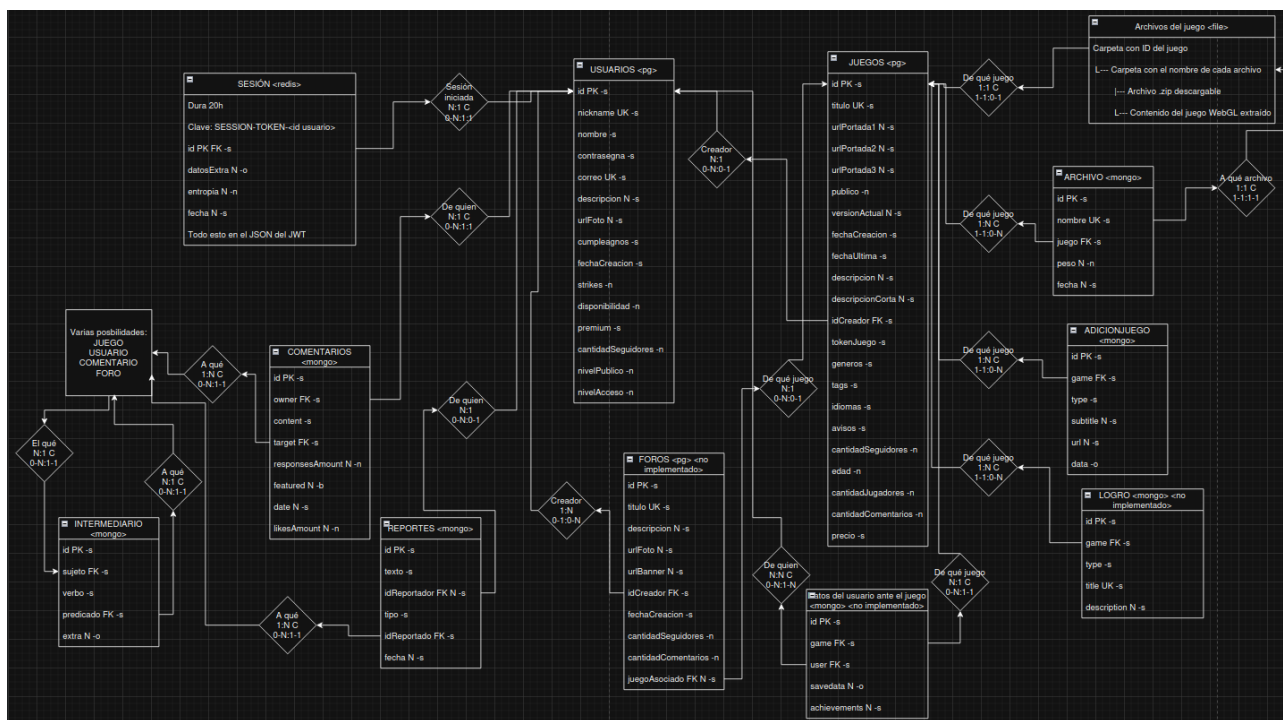


Diagrama UML de las bases de datos (previsualización)

Muchas de estas entidades luego tienen una clase (que en realidad es un Type) en el backend y frontend

Definición de entidades

A continuación se da una breve explicación de cada una de las entidades y sus propiedades, para ver información sobre las operaciones que se hacen con estas entidades consultar la documentación de la API.

USUARIOS

Se trata de la entidad principal de la base de datos, sobre la que giran el resto. Almacenaría todos los datos del usuario y su perfil, y como se requiere una robustez y estructura rígidas se almacena en Postgres.

- **id**: Clave primaria string, UUID que identifica a la entidad.
- **nickname**: Propiedad única string, nombre URL-friendly del usuario, entre 4 y 15 caracteres, letras mayúsculas o minúsculas del alfabeto anglosajón, cifras arábigas y "-", "_" o "|". Se puede usar para iniciar sesión.
- **nombre**: String, nombre formal del usuario que admite más caracteres incluyendo el espacio y es entre 5 y 100 caracteres.

MEDIASTRAY - Eloy Rico Payá

- **contraseña**: String, lo que se almacena es el hash de la contraseña usando BCrypt, pero la contraseña debe tener mínimo 8 caracteres, mayúsculas, minúsculas, símbolos y cifras. Es obligatoria para iniciar sesión.
- **correo**: Propiedad única, string, almacena la dirección de correo electrónico del usuario, se puede usar para iniciar sesión pero por limitaciones del servicio de correo no se puede comprobar que ese correo sea real y que pertenezca al usuario.
- **descripcion**: Nullable, string, almacena hasta 512 caracteres de descripción del usuario. Si el usuario tuviese Magna aquí se guardaría el Markdown.
- **urlFoto**: Nullable, string, almacena una URL válida que indica la imagen de la foto de perfil, se hace así ya que la app no ofrece almacenamiento multimedia.
- **cumpleaños**: String, almacena el timestamp de la fecha de nacimiento del usuario, se usa para bloquear los juegos que requieran cierta edad aunque también puede ser útil para algunos análisis de mercado.
- **fechaCreacion**: String, almacena el timestamp de la fecha en la que se creó la cuenta.
- **strikes**: Número entero, es un valor a modo de flag usado por los administradores para marcar usuarios.
- **disponibilidad**: Número entero, permisos restrictivos que cuanto más aumentan más restringido está el usuario. 0 = normal, 1 = no puede subir juegos, 2 = no puede interactuar, 3 = totalmente deshabilitado.
- **premium**: String, almacena la fecha de caducidad del Magna de un usuario, si está atrasado de la fecha actual o está vacío es que no tiene Magna.
- **cantidadSeguidores**: Número entero, almacena la cantidad de seguidores para no tener que recalcularla.
- **nivelPublico**: Número entero, el nivel de anonimato del usuario. 0 = normal, 1 = los otros usuarios pueden saber que existe pero no pueden ver la mayoría de sus datos, 2 = totalmente anónimo.
- **nivelAcceso**: Número entero, define los permisos. 0 = normal, 1 = panel de administración básico, 2 = usuario sudo (permisos totales), 3 = permisos de moderador (aún más reducidos que los de administrador)

JUEGOS

Es junto a usuario la entidad principal de la base de datos, almacena el perfil de un juego y de este dependen muchas informaciones adicionales.

- **id**: Clave primaria string, UUID que identifica a la entidad.
- **título**: Clave única, string, indica el título del juego entre 3 y 64 caracteres.
- **urlPortada1**: String, URL válida para la portada del juego en 460x215px.
- **urlPortada2**: String, URL válida para la portada del juego en 600x900px.
- **urlPortada3**: String, URL válida para la portada del juego en 1920x1080px, se usa también como fondo.
- **publico**: Booleano, si es false solo el creador del juego y los administradores pueden verlo.
- **versionActual**: String, indica el tag de la versión actual del juego, es hasta 16 caracteres y debe seguir "<lo que sea><cifra>.<cifra>.<cifra opcional><lo que sea>".
- **fechaCreacion**: String, es el timestmap con la fecha en la que se creó el juego en la app.
- **fechaUltima**: String, es el timestamp de la fecha de la última modificación del juego por parte del creador.
- **descripcion**: Nullable, string, se trata de la descripción larga del juego, almacena hasta 8192 caracteres y si su creador tiene Magna aquí se guardaría el Markdown.

MEDIASTRAY - Eloy Rico Payá

- **descripcionCorta**: Nullable, string, descripción corta del juego para otros contextos de hasta 128 caracteres y sin Markdown.
- **idCreador**: Clave ajena, string, es el id del usuario que ha creado el juego, no hay borrado en cascada así que si el usuario se borra el juego sigue estando (por cuestiones de preservación del videojuego)
- **tokenJuego**: String, parámetro para uso interno, se usaría también en el caso de implementar la API de logros y guardado en la nube (cosa que no dio tiempo)
- **generos**: String, lista de palabras separadas por coma de hasta 256 caracteres donde cada una es un género del juego.
- **tags**: String, lista de palabras separadas por coma de hasta 256 caracteres donde cada una es un tag del juego.
- **idiomas**: String, lista de palabras separadas por coma de hasta 256 caracteres donde cada una es un idioma disponible en el juego en formato ISO.
- **avisos**: String, lista de palabras separadas por coma de hasta 256 caracteres donde cada una es un aviso antes de jugar el juego.
- **cantidadSeguidores**: Número entero, guarda la cantidad de seguidores del juego para no tener que recalcularla
- **edad**: Nullable, número entero, es la edad mínima que tiene que tener el usuario para jugar o ver el juego, esto se comprueba en base al cumpleaños. Si es 0 entonces no hay restricción
- **cantidadJugadores**: Número entero, guarda la cantidad de jugadores del juego para no tener que recalcularla. Se entiende como jugador un usuario (con cuenta) que ha entrado en la página del juego.
- **cantidadComentarios**: Número entero, guarda la cantidad de comentarios del juego para no tener que recalcularla.
- **precio**: Nullable, string, almacena la etiqueta de precio del juego (si se implementa un sistema de pagos real esto sería distinto). Esta funcionalidad solo está disponible para usuarios con Magna aunque luego el método de pago no esté implementado. Debe seguir este patrón en caso de estar presente: <caracteres><cifras>.<cifras><caracteres>, tiene que haber mínimo un carácter que sería el símbolo de la moneda.

COMENTARIOS

Objetos escribibles por los usuarios hacia otros objetos como juegos, foros o incluso otros comentarios (lo que se llamaría respuestas), al ser más versátiles, necesitar mayor velocidad y ser menos cruciales se almacenan en MongoDB en su propia colección.

- **id**: Clave primaria string, UUID que identifica a la entidad.
- **owner**: Clave ajena, string, almacena el UUID del usuario que lo ha creado.
- **content**: String, texto del comentario de hasta 512 caracteres, en caso de que el usuario tenga Magna aquí se almacenará Markdown.
- **target**: Clave ajena, string, almacena el UUID del objeto al que hace referencia el comentario, que puede ser otro comentario, un juego o un foro.
- **responsesAmount**: Número entero, guarda la cantidad de comentarios directos a este comentario para no tener que recalcularla
- **featured**: Nullable, booleano, si es true tendrá preferencia al salir primero, los comentarios de los usuarios con Magna son featured.

MEDIASTRAY - Eloy Rico Payá

- **date**: Nullable, string, almacena el timestamp con la fecha en la que fue creado el comentario.
- **likesAmount**: Número entero, cantidad de likes que tiene el comentario.

REPORTES

Los reportes son objetos de moderación, un usuario (tenga cuenta o no) puede reportar un objeto (como un usuario, juego, comentario, foro) y dar una razón por ello para que los moderadores actúen, ya que estos son los únicos que pueden ver los reportes. El mismo usuario no puede reportar dos veces el mismo objeto. Los reportes se almacenan en MongoDB.

- **id**: Clave primaria string, UUID que identifica a la entidad.
- **texto**: String, texto con la razón del reporte. Admite hasta 256 caracteres.
- **idReportador**: Nullable, clave ajena, string, hace referencia al UUID del usuario que reporta, en caso de ser reportado por una persona sin cuenta saldrá como nulo.
- **tipo**: String, almacena como palabra el tipo de objeto reportado ("game", "user", "comment", "forum"), usado para filtrar.
- **idReportado**: Clave ajena, string, es el UUID del objeto reportado.
- **fecha**: Nullable, string, timestamp con la fecha en la que se lanzó el reporte

INTERMEDIARIO

Es un objeto creado para representar una relación, por ejemplo que un usuario sigue un juego, que ha dado un like, etc. Funciona a modo de sujeto - verbo - predicado y también se almacena en MongoDB. Es necesario para almacenar que ha ocurrido algo.

- **id**: Clave primaria string, UUID que identifica a la entidad.
- **sujeto**: Clave ajena, string, es el UUID de quien realiza la acción, la mayoría de las veces un usuario.
- **verbo**: String, que tipo de acción se realiza: sigue, like, juega, pertenece.
- **predicado**: Clave ajena, string, UUID del objeto al que se le realiza la acción, como un usuario, juego, comentario, etc.
- **extra**: Nullable, objeto, almacena posibles datos extra de la acción, como una fecha.

ARCHIVO

En este caso puede hacer referencia tanto a los archivos físicos del juego almacenados en el servidor o al objeto de cabecera almacenado en MongoDB para tener un rastro de estos archivos (y no tener que realizar acciones de disco cada vez que se consulta la información de un juego). Refiriéndose a los archivos:

En una ruta concreta cada juego tiene un directorio cuyo nombre es el UUID, y dentro se encuentran subcarpetas cada una con el nombre del archivo asociado a ese juego, dentro de estas se puede encontrar un .zip o los archivos del juego WebGL. Los usuarios con Magna pueden tener más archivos y más pesados.

Refiriéndose al objeto en MongoDB:

- **id**: Clave primaria string, UUID que identifica a la entidad.
- **nombre**: Clave única (dentro del mismo juego), string, es la etiqueta identificativa del archivo en el juego. Tiene que ser un nombre de archivo posible en Windows y Linux

MEDIASTRAY - Eloy Rico Payá

- **juego**: Clave ajena, string, indica el UUID del juego al que hace referencia.
- **peso**: Nullable, número entero, el peso en bytes del archivo, para no tener que recalcularlo cada vez.
- **fecha**: Nullable, string, timestamp con la fecha en la que se publicó el archivo dentro del juego.

ADICIÓN JUEGO

Es información adicional del juego almacenada en MongoDB como pueden ser imágenes, el trailer, requisitos de hardware, etc. El mismo juego puede tener varias, pero los usuarios con Magna pueden tener más en sus juegos.

- **id**: Clave primaria string, UUID que identifica a la entidad.
- **game**: Clave ajena, string, indica al UUID del juego al que hace referencia.
- **type**: String, tipo de adición, su comportamiento cambiará dependiendo de este. Los disponibles son: trailer, images, site, ost, requirements, event, text y mention.
- **subtitle**: Nullable, string, texto adicional para la adición, hasta 32 caracteres.
- **url**: Nullable, string, URL válida para acompañar a dicha información.

- **data**: Objeto, almacena la información de la adición, su contenido varía dependiendo de type:

Caso trailer: data.**iframe**: String que almacena el iframe del video, útil para Youtube, hasta 430 caracteres.

Caso site: data.**icon**: String que almacena la URL del ícono de la adición.

Caso ost: data.**cover**: String que almacena la URL de la portada del OST.

Caso images: data.**images**: Array de hasta 32 URLs que representan las imágenes del carrusel (los usuarios sin Manga pueden hasta 10)

Caso requirements: data.**specs**: String de hasta 1024 caracteres con los requisitos de hardware del juego.

Caso event: data.**image**: String con la URL de la imagen del evento. data.**info**: String con el texto de la información del evento, hasta 128 caracteres.

Caso text: data.**text**: String con texto auxiliar para distintos usos, hasta 64 caracteres.

Caso mention: data.**nickname**: String que sería el nickname del usuario al que se le quiere hacer mención, sigue las mismas normas de validación que el nickname convencional.

MEDIASTRAY - Eloy Rico Payá

A partir de aquí se mencionan las entidades de la base de datos que no se pudieron implementar por falta de tiempo. La información que hay sobre estos en el diagrama UML es solo conceptual.

FOROS

Similar a como los comentarios interactúan con el juego, los foros serían espacios a propósito para esto con algo de información de perfil (como un banner de fondo). Además podrían ser privados para ciertos usuarios, y estarían enlazados a un juego existente, ya sea de Mediastray o de IGDB.

LOGRO

A los usuarios con Magna se les daría la posibilidad de configurar logros de varios tipos en sus juegos, los cuales se integrarían en estos mediante un SDK.

DATOS GUARDADO

Siguiendo la lógica del SDK de los logros, también se podría hacer que por cada usuario en cada juego no solo se guarde que logros tiene dicho usuario, sino también guardado en la nube, por ejemplo un JSON que guarde la partida del jugador.

2.4 Listado y justificación de las herramientas en frontend

Tecnologías en el frontend

Como framework para desarrollar el frontend se usó React usando NPM como manejador de librerías y Vite como manejador de proyecto, todo con TypeScript. También se utilizaron múltiples librerías (además del propio React)

- **ReactDOM**: Utilizado para el manejo de rutas y algunas acciones del DOM
- **Capacitor**: Utilizado para generar un APK del frontend (aunque por ahora sigue algo inestable)
- **Electron**: Utilizado para generar un ejecutable para Linux y Windows del frontend (también es algo inestable)
- **ReactErrorBoundary**: Utilizado para los errores con las rutas
- **ReactRouter**: Utilizado también para las rutas
- **ReactHookForm**: Handler para formularios reactivos con estados
- **RemarkGFM** y **ReactMarkdown**: Ambos utilizados para el renderizador de Markdown
- **TailwindCSS**: Librería para usar Tailwind de manera optimizada (dejando solo las clases usadas en la dist)
- **Zod**: Funciones de validaciones

Y estas son las librerías usadas para el desarrollo

- Dependencias de **Capacitor**, **Electron**, **Typescript** y **Vite**
- **Eslint** y dependencias: Utilizado para formateos de código

En el package.json también se definen algunos comandos para automatizaciones y CI/CD así como configuraciones para hacer la build.

Arquitectura empleada

Se ha aplicado una arquitectura bastante estándar teniendo en cuenta qué se suele seguir a la hora de desarrollar con React puro. Se intenta que los componentes sean lo más reutilizables y desacoplables posible.

Por otro lado hay cuatro contextos en la app, uno de ajustes que devuelve datos básicos como la URL de la API o la API Key, el idioma, etc. Otro contexto para mensajes flotantes que permite lanzar nuevos y gestionar los que están por desaparecer. Otro para las sesiones que ofrece información sobre el usuario actual y funciones para cambiar la sesión. Y otro contexto para los juegos y las operaciones relacionadas con este.

Muchas entidades (como los juegos) no se almacenan en el contexto porque, por mucho que esto ahorraría muchas consultas a la API, se considera a los juegos como una entidad cambiante que de un momento a otro podría desaparecer o cambiar sus datos, por eso en la mayoría de casos es mejor que se vuelva a llamar a la API. Lo mismo pasa por ejemplo con los comentarios o con otros usuarios (sin embargo los datos del usuario actual sí que se guardan en el contexto)

En cuanto a los hooks, se ha creado un hook con las funciones para interactuar con cada entidad de la API (comentarios, juegos, usuarios, operaciones de admin), hooks para acceder a los contextos antes mencionados, un hook para interactuar con el LocalStorage (usado para guardar ajustes como el idioma o el token de sesión actual), otro hook para los títulos HTML y otras interacciones del DOM, y otro hook para el idioma, que devuelve un texto traducido concreto al idioma actual.

MEDIASTRAY - Eloy Rico Payá

Además, a lo largo de la aplicación se han aplicado algunas optimizaciones en los componentes como los hooks (ya existentes en React) como `useCallback` y `useMemo`, además de la función `memo`. Esto permite un lazy loading que hace que la aplicación cargue más rápido en muchos casos (por ejemplo a pesar de que React esté pensado para SPA, de esta manera no se carga la funcionalidad de todas las páginas de golpe ya que estas optimizaciones también se aplicaron en el enrutador)

Algo a resaltar del frontend es su manejo de idiomas, existe un ajuste que almacena el código ISO del idioma actual y dentro de los componentes, en lugar de escribirse los textos directamente se llama al hook de idioma indicándole la sección y el nombre del texto concreto para que renderice, entonces el hook buscará en un JSON con todos los textos de la app en varios idiomas (`textosInterfaz.json`) para devolver el texto adecuado en el idioma actual.

Sobre la reutilización de componentes, se intentó englobar funcionalidades dentro de componentes desacoplables, por ejemplo en lugar de utilizar la etiqueta HTML de `input`, se hizo una que engloba además al `select` y al `textarea` y que es totalmente compatible con textos de guía, mensajes de error, validaciones y `ReactHookForm`. Estrategias similares se aplican en componentes para botones, enlaces, etc.

En cuanto al acceso a la API, existen funciones (en sus debidos hooks) para cada endpoint de la API (aunque en algunos casos hay varias funciones por endpoint o varios endpoints por función), estas funciones recibirían y validarían los datos a la hora de lanzar la petición y devolverían el resultado de la API. Además estos hooks tienen un estado de error y cargando para usarlo en los componentes. Estas funciones se usan directamente en los componentes o en los contextos.

2.5 Servicios utilizados y proceso de despliegue

Para el despliegue y funcionamiento se hizo uso de Docker (con la ayuda de Docker Desktop y planteando haber usado Kubernetes)

Arquitectura de microservicios

La app consta de 5 contenedores de Docker desplegados mediante Docker Compose, todo dentro de la misma red en modo bridge y

- **frontend**: Este es opcional pero desplegaría un servidor de Nginx para el frontend.
- **backend**: El contenedor principal, ejecuta el servidor de NodeJS y se conecta con el resto de contenedores ofreciendo además la API. También es capaz de servir el frontend bajo cierta configuración.
- **redis**: La base de datos de Redis utilizada por el backend.
- **mongodb**: La base de datos de MongoDB utilizada por el backend, protegida con credenciales.
- **postgresql**: La base de datos de PostgreSQL utilizada por el backend, protegida con credenciales.

Algunos de estos contenedores copian archivos de la máquina host o enlazan directorios, por ejemplo el backend copiaría todos los scripts generados del backend, el frontend copiaría el dist, las bases de datos Postgres y Mongo se guardan en la máquina host, etc.

Por otra parte, el backend (que sería el corazón de la infraestructura) es totalmente desacoplable al resto de microservicios, se podrían tener las bases de datos, los archivos o el frontend en otro servidor si se configura el .env de manera adecuada.

Funcionalidades CI/CD y automatizaciones

Hay dos maneras de desplegar la app, una es en el entorno de desarrollo con el script dev.sh, esta lanzaría los contenedores (menos el de frontend) con docker-compose-dev.yml, lo cual implica que se activa el modo debug o se permite el hot-reload (cuando un archivo de código cambia su contenido durante la ejecución, los cambios se ven aplicados en el backend o frontend), y las bases de datos están accesibles desde fuera.

Si por el contrario se despliega la app mediante prod.sh se desplegará un entorno listo para producción, usando las build finales, sin hot-reload ni debug y con las bases de datos inaccesibles desde fuera.

Por el tamaño del proyecto no se implementó una pipeline CI/CD como podría ser el despliegue automático de la rama main, pero sí que se usaron estas librerías de hot-reload así como tests automáticos con Bruno para la API.

En cuanto a las automatizaciones, hay un script para limpiar toda la base de datos y dejar el proyecto como nuevo, otro para lanzar la app en modo desarrollo, otro para modo producción y otro para cerrarlo todo, estos scripts son de Bash en formato .sh y se encuentran principalmente en la raíz del proyecto.

Tecnologías de despliegue

Como se explica antes, si bien se usa Docker también se puede separar el despliegue en servidores distintos, lo que haría falta es o bien un entorno Docker en Linux con recursos suficientes, o un entorno con NodeJS, un servidor de archivos estáticos (para el frontend, multimedia y los juegos) y las bases de datos PostgreSQL, MongoDB y Redis.

En el caso de desplegarse en un IAAS como AWS se podría optar por el despliegue en Docker directamente en un EC2 (opción más barata pero mínima), usar el servicio ECS, o tenerlo todo por separado, por ejemplo corriendo el backend en un EC2, los juegos en un S3, el frontend servido por Cloudflare, y las bases de datos cada una en un EC2, o incluso modificando el código existente se podrían usar bases de datos propias de AWS como RDS, DynamoDB y Valkey. Por supuesto configurando la red para ello.

MEDIASTRAY - Eloy Rico Payá

Pasos para desplegar

Veamos ahora los pasos para desplegar la app, partiendo de que ya tenemos el repositorio clonado. Sin importar que opción escojamos lo primero será configurar el .env. En este caso basta con editar .env.example y frontend.env.example ya que su contenido se copiará a los .env de verdad al ejecutar el script de despliegue.

Hay dos archivos .env, esto es lo que podemos encontrar en el de frontend (frontend.env.example)

- **REACT_APP_API_URL**: URL de la API (el backend)
- **REACT_APP_PUBLIC_URL**: URL con los archivos públicos (donde se encuentra la multimedia, por defecto ya la sirve el propio backend)
- **REACT_APP_GAMES_URL**: URL donde se sirven los juegos, por defecto también lo hace el backend.
- **REACT_APP_API_KEY**: Misma API key usada para ofuscar ciertas rutas del backend, se elige en el otro .env.
- **REACT_APP_TAMAGNO_PAGINA**: Tamaño de paginado a la hora de organizar múltiples datos.
- **REACT_APP_FRECUENCIA_ACTUALIZACION**: Cantidad de milisegundos que tardan ciertas peticiones en volver a hacerse, como por ejemplo el buscador.
- Las mismas propiedades otra vez pero reemplazando "REACT_APP_" por "VITE_"

Y este sería el .env para el backend y las bases de datos

- **PUBLIC_FILES_PATH**: Localización de los archivos públicos, en caso de servirlos.
- **GAMES_FILES_PATH**: Localización de los archivos de los juegos, en caso de servirlos.
- **FRONTEND_DIST_PATH**: Localización de los archivos del frontend, en caso de servirlos.
- **INIT_TESTS**: True para ejecutar los tests de conexiones al iniciar el backend.
- **INIT_METRICS**: True para exponer las métricas de rendimiento para el monitoreo.
- **SERVE_FRONTEND**: True para que el backend sirva los archivos del frontend también.
- **SERVE_STATIC**: True para que el backend sirva los archivos públicos y de juegos también.
- **FRONTEND_URL**: Por cuestiones de CORS, URL del frontend que hará las peticiones.
- **FRONTEND_URL_DEV**: Por cuestiones de CORS, URL del frontend que hará las peticiones (para dev).
- **API_PRIVATE_TOKEN**: Token para ofuscar ciertos endpoints, será necesario en el frontend.env.
- **JWT_SECRET**: Contraseña del cifrado JWT.
- **FREE_CORS**: True para no limitar por CORS.
- **RUTA_LOGS**: Directorio donde guardar los logs del backend.
- **TIEMPO_LOGS**: Tiempo en milisegundos tras los que se tiene que escribir la información de los logs.
- **DURACION_SESSION**: Duración de la sesión de usuario.
- **DURACION_SESSION_TOKEN**: Duración de la sesión del usuario (concretamente el token)
- **TAMAGNO_PAGINA**: Tamaño de página estándar para las operaciones con paginado.
- **MAX_TAMAGNO**: Tamaño máximo en Gb que se permite al subir un archivo.
- **POSTGRES_USER**: Usuario para la base de datos de Postgres
- **POSTGRES_PASSWORD**: Contraseña para la base de datos de Postgres

MEDIASTRAY - Eloy Rico Payá

- **POSTGRES_DB**: Nombre de la base de datos en el SGDB de Postgres.
- **DATABASE_URL**: URL de conexión a Postgres.
- **SQL_INIT_PATH**: Localización donde dejar los archivos de la base de datos Postgres.
- **SQL_INIT_PATH_FAKES**: Localización con el script de datos de ejemplo SQL para dev (deprecated)
- **MONGO_INITDB_ROOT_USERNAME**: Usuario para la base de datos de Mongo.
- **MONGO_INITDB_ROOT_PASSWORD**: Contraseña para la base de datos de Mongo.
- **MONGODB_URI**: URL para la conexión a Mongo.
- **REDIS_HOST**: URL para la conexión a Redis
- **DOMAIN**: Nombre de dominio que se usará, útil para CORS.
- **BACKEND_PORT**: En que puerto se servirá la API (habría que cambiarlo en más sitios)

Después de configurar los .env el siguiente paso sería exponer el puerto que vallamos a usar para que sea accesible, y ejecutar prod.sh que desplegará los contenedores de Docker necesarios, y cuando termine ya estaría listo.

Para desplegarlo en modo desarrollo hay que tener NodeJS y NPM instalados, pero si lo queremos desplegar de una manera distinta que con Docker hay que ejecutar el servidor con "node server.js" tras convertir el TypeScript en JavaScript (el resultado se almacena en dist), servir los archivos generados por el frontend también con algún servidor web, lo mismo con los archivos públicos y los juegos y tener la conexión preparada para las bases de datos PostgreSQL, MongoDB y Redis.

Despliegue actual

A continuación se explicará la arquitectura empleada para desplegar la app en AWS, es importante mencionar que actualmente es un entorno de demostración y no estaría preparado para escalar masivamente (aunque se explicará como)

<HACER>

Monitoreo y testing

El proyecto, aunque de forma muy sencilla, dispone de tests y monitoreo automático.

En cuanto al test, se centra en un script adicional ejecutable de manera opcional al iniciar el backend que hace algunas transacciones de prueba con las conexiones del servidor (bases de datos, archivos, logs) y muestra mensajes de éxito o error. No se han usado tests unitarios como se suele hacer por ejemplo con Java ya que se optó por un método de testing más tradicional, probando peticiones con Bruno (el cual tiene una colección aposta para el testing) y probando la interfaz manualmente.

En cuanto al monitoreo, en el .env hay una opción para hacer que el backend exponga métricas de rendimiento a Grafana y Prometheus, los cuales se pueden ejecutar también con un docker-compose-monitorize.yml, sin embargo esto no ha sido tan usado ya que para este tipo de app suele ser mejor consultar el rendimiento con Docker, o en caso de estar desplegado con las propias opciones del IAAS.

2.6 Herramientas externas para el desarrollo

Además de todas las tecnologías y herramientas listadas anteriormente, en este apartado se mencionan otras que no son tecnologías integrales que afecten a la aplicación sino que se usaron para facilitar su desarrollo.

- Se usó el IDE VSCode, con una gran cantidad de extensiones (más de 100) para formateo de código, facilidades con Docker, modo debug, etc.
- Se usó Linux (distribución propia) para facilitar el desarrollo y poder hacer uso de comandos y scripts de automatización de Bash (como prod.sh que lanzaría la app automáticamente)
- Además de Docker Engine, se usó Docker Desktop para un mejor manejo de los contenedores
- Como cliente de API se usó Bruno, las colecciones de las peticiones están también incluidas en el repositorio
- Como cliente de base de datos se usó Mongosh para MongoDB y DBeaver para Postgres
- Se usó Obsidian como app organizativa con listas de objetivos, diagramas de tiempo, etc.
- Se hizo uso de diferentes navegadores para hacer testing en el frontend
- Como herramientas de mockup y documentación se usaron Penpot, DrawIO, Excalidraw, Gimp y LibreOffice Writer
- Para agilizar el desarrollo se hizo uso de la IA, concretamente el modelo Qwen ejecutándose en LMStudio

3 Guía de la app

Explicación de las distintas partes y funciones de la app

3.1 Validaciones de campos

Requisitos para los datos (por ejemplo que la contraseña debe contener X, o que el nickname no puede ser Y, o que un juego puede tener hasta Z adiciones...)

3.2 Cuentas Magna

Explicación en términos de marketing de la cuenta Magna y las ventajas que ofrece

3.3 Publicación de juegos

Pasos y consideraciones a la hora de subir un juego a la app

3.4 Moderación y normas

Explicación de como se modera la app, las normas y sobre el sistema de admin

3.5 Algoritmo de recomendación

Explicación de funcionalidades como el orden de comentarios, juego diario y semanal, juegos destacados

4 Ampliaciones y líneas de mejora

Funcionalidades más secundarias que habría estado bien incluir en la app pero no se hizo por falta de tiempo (como los foros o la API de logros)

Como funcionaría si se usase en el mundo real

5 Conclusiones y comentarios

Reflexión y aprendizaje

Referencias bibliográficas

Acceso a las fuentes de información como documentaciones empleadas o IAs utilizadas